МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
КРЕМЕНЧУЦЬКИЙ НАЦІОНАЛЬНИЙ
УНІВЕРСИТЕТ ІМЕНІ МИХАЙЛА
ОСТРОГРАДСЬКОГО
НАВЧАЛЬНО-НАУКОВИЙ ІНСТИТУТ ЕЛЕКТРИЧНОЇ
ІНЖЕНЕРІЇ ТА ІНФОРМАЦІЙНИХ ТЕХНОЛОГІЙ

Кафедра автоматизації та інформаційних систем

Освітній компонент

«АЛГОРИТМИ І СТРУКТУРА ДАНИХ»

ЗВІТ
З ПРАКТИЧНОЇ РОБОТИ № 4

виконав:
студент групи КН-25-1
Заїченко А.С.
Перевірив:
Старший викладач кафедри АІС
Сидоренко В.М.

Кременчук 2026

Практична робота № 4

Тема. Алгоритми пошуку та їх складність

Мета: опанувати основні алгоритми сортування та навчитись методам аналізу їх асимптотичної складності.

Порядок виконання роботи

1. Оцінити асимптотичну складність алгоритму лінійного пошуку у *O*- нотації в найгіршому і в найкращому випадку. Як можна покращити алгоритм лінійного пошуку?

Лінійний пошук:

- O(1) - найкращий випадок
- O(n) - найгірший випадок

Покращення:

- бінарний пошук (якщо масив відсортований)
- хеш-таблиці
- sentinel-оптимізація

2. Оцінити асимптотичну складність алгоритму бінарного пошуку у *O*- нотації в найгіршому і в найкращому випадку.

Бінарний пошук:

- Найкращий випадок: O(1)
- Найгірший випадок: O(log n)

Бінарний пошук набагато швидший за лінійний O(n), але потребує відсортованих даних.

3. Побудувати алгоритм тернарного пошуку і оцінити його асимптотичну складність алгоритму у *O*-нотації **в найгіршому і в найкращому випадку. Який з алгоритмів є оптимальнішим: бінарний, чи тернарний? Обґрунтувати відповідь відповідними обчисленнями.**

Тернарний пошук:

```python
def ternary_search(arr, left, right, x):
    if right < left:
        return -1

    m1 = left + (right - left) // 3
    m2 = right - (right - left) // 3

    if arr[m1] == x:
        return m1
    if arr[m2] == x:
        return m2

    if x < arr[m1]:
        return ternary_search(arr, left, m1 - 1, x)
    elif x > arr[m2]:
        return ternary_search(arr, m2 + 1, right, x)
    else:
        return ternary_search(arr, m1 + 1, m2 - 1, x)
```

Рисунок 1 - Алгоритм тернарного пошуку

Найкращий випадок:

$0(1)$

Найгірший випадок:

$0(log_3 n) \approx 0(log\, n)$

Бінарний пошук виконує приблизно $log_2 n$ кроків $i \sim 1$ порівняння на кожному кроці, тому $T_{bin}(n) \approx log_2 n$.

Тернарний пошук виконує $log_3 n \approx 0.63 log_2 n$ кроків, але ~ 2 порівняння на крок, тому $T_{ter}(n) \approx 2log_3 n \approx 1.26 log_2 n$.

Тому бінарний пошук ефективніший на практиці, хоча мають однакову складність.

4. Порівняти ефективність алгоритмів лінійного, бінарного та тернарного пошуку для різних розмірів вхідного списку. Для цього провести 30 експериментальне дослідження та побудувати графіки залежності часу виконання алгоритму від розміру вхідного списку.

Алгоритм	Складність
Лінійний	O(n)
Бінарний	O(log n)
Тернарний	O(log n)

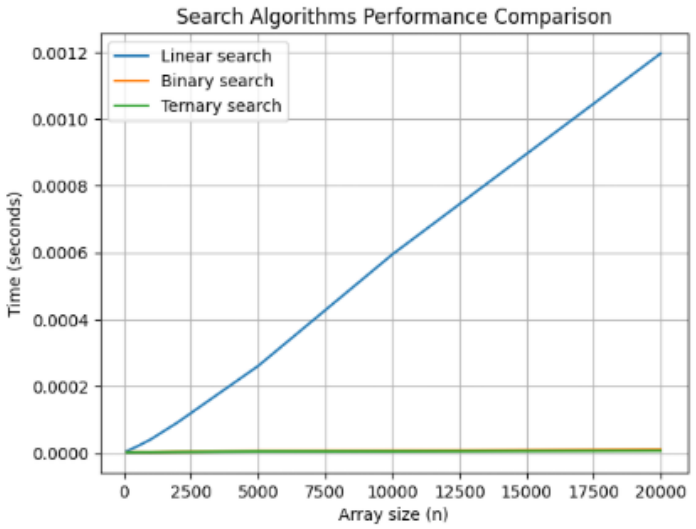

Рисунок 2 - Графік

- Лінійний пошук росте майже прямо → підтверджує O(n)
- Бінарний і тернарний майже “лежать” біля нуля → логарифмічна складність O(log n)
- Тернарний трохи повільніший за бінарний через більшу кількість порівнянь

5. Порівняти алгоритми пошуку за їхньою здатністю працювати з відсортованими та не відсортованими списками. Провести аналіз впливу відсортованості списку на час виконання кожного алгоритму.

Алгоритм	Невідсортований список	Відсортованийсписок	Вплив
Лінійний	O(n)	O(n)	Слабкий
Бінарний	-	O(log n)	Критичний
Тернарний	-	O(log n)	Критичний

1. Лінійний пошук майже не залежить від сортування (лише невелика оптимізація).
2. Бінарний і тернарний пошук повністю вимагають відсортованого списку.
3. Відсортованість:

- не дає великого ефекту для O(n)
- але радикально покращує O(log n) алгоритми

6. Розглянути сценарії використання кожного з алгоритмів пошуку у практичних задачах і обґрунтувати вибір кожного алгоритму в конкретному випадку.

Вибір алгоритму пошуку напряму залежить від структури даних:

- Якщо дані не відсортовані → використовується лінійний пошук.
- Якщо дані відсортовані та великі → оптимальним є бінарний пошук.
- Тернарний пошук застосовується у вузькоспеціалізованих задачах оптимізації, але рідко використовується у звичайних програмах.

Контрольні питання

Що таке алгоритм пошуку і чому він важливий у контексті комп'ютерних наук?

Алгоритм пошуку — це набір правил і дій, які дозволяють знайти потрібний елемент у структурі даних (масиві, списку, базі даних тощо).

Які основні критерії оцінки ефективності алгоритмів пошуку?

Основні критерії:

- **Часова складність** (швидкість роботи алгоритму)
- **Просторова складність** (використання пам’яті)
- **Кількість порівнянь**
- **Стійкість до розміру даних**
- **Необхідність попередньої обробки**

Що таке лінійний пошук, і як він працює?

Лінійний пошук — це метод, при якому елементи перевіряються послідовно один за одним.

Як працює:

- починає з першого елемента
- порівнює кожен елемент із шуканим
- зупиняється, коли знайде або досягне кінця списку

Які умови повинні бути виконані для успішного застосування бінарного пошуку?

Для роботи бінарного пошуку потрібно:

- список **повинен бути відсортований**
- елементи повинні бути доступні за індексом (масив або подібна структура)
- можливість порівняння елементів

Які переваги та недоліки використання бінарного пошуку порівняно з іншими алгоритмами пошуку?

Переваги:

- дуже швидкий: O(log n)
- значно менше порівнянь, ніж у лінійному пошуку

- ефективний для великих обсягів даних

Недоліки:

- працює тільки з відсортованими даними
- складніший у реалізації, ніж лінійний пошук
- потребує попереднього сортування (якщо дані не відсортовані

Що таке тернарний пошук, і в чому його відмінність від бінарного пошуку?

Тернарний пошук — це алгоритм, який ділить відсортований масив на три частини замість двох.

Відмінності:

- бінарний пошук → ділить на 2 частини
- тернарний пошук → ділить на 3 частини

Складність:

- також O(log n), але з більшою кількістю порівнянь

Особливості:

- використовується рідше
- ефективний у задачах оптимізації та пошуку максимумів/мінімумів функцій